

XRAG Freeze-Ingest Driver Specification (XRAG-only)

Freeze-Ingest Contract v1.0.0 — Driver v1.0.0

Prepared for XRAG Producer Integration

March 3, 2026

1 Scope and Goals

This document specifies the external **XRAG Freeze-Ingest Driver** interface and acceptance rules for **XRAG Freeze Packages**. The driver is:

- **Stateless** (no persistence, no cross-run memory).
- **Deterministic** (same input bytes $\Rightarrow$ same decision and hashes).
- **Single-run** (one package per invocation).
- **Read-only validation** (no ontology/physics interpretation).
- **Schema + frozen-discipline + bounded-grammar enforcement**.
- **Hash-stable audit record** (canonicalization + SHA-256).

Non-goals:

- No engine execution (the driver does *not* run any reasoning engine).
- No semantic correctness checks of the proposed axiom.
- No access to producer repos or internal XRAG state.

2 Contract Versions

- **Driver version:** `xrag-driver/1.0.0`
- **Contract version:** `freeze-ingest-contract/1.0.0`
- **Schema file:** `xrag_schema.json` (JSON Schema 2020-12)

3 Inputs

3.1 Transport

The driver reads a single JSON document from **STDIN**. The input must be UTF-8 JSON.

3.2 Message Types

The producer must set exactly one of:

- `message_type = "candidate_axiom"`
- `message_type = "frozen_axiom"`

3.3 Required Top-Level Fields

A valid XRAG Freeze Package includes:

- `message_type`
- `package_id`
- `created_at` (RFC3339 / date-time)
- `producer { name, version, optional build }`
- `payload { record_type, record_id, record }`
- `provenance { source, chain }`
- `integrity { hash_alg }`
- `protocol` (must be frozen)
- `comparator` (must be frozen)
- `bridge_rules` (must be frozen)

3.4 Frozen Discipline (Authority Separation)

- **Candidate:** `candidate_axiom` packages may represent tentative content.
- **Frozen:** `frozen_axiom` packages must include a non-empty `provenance.chain`.

Minimum enforcement:

- For `frozen_axiom`: `provenance.chain` must contain at least one step. Recommended: include an `expert_reviewed` step with reviewer identity and optional note.

4 Bounded Grammar and Capacity Budgets

4.1 Capacity Budgets (Fail-Closed)

The driver rejects pathological inputs by enforcing finite budgets:

Budget	Default
--------	---------

MAX_RAW_BYTES	2,000,000 (2 MB)
MAX_NESTING_DEPTH	32
MAX_KEYS_PER_OBJECT	256
MAX_ARRAY_LEN	2048
MAX_STRING_LEN	4096
MAX_PARAMETERS	64
MAX_UNITS	64
MAX_EXPRESSION_LEN	2048

4.2 Parameter Name Rule

All parameter keys in `payload.record.parameters` must match:

```
^[a-zA-Z][a-zA-Z0-9_]{0,63}$
```

4.3 Expression Safety (Programmable Forcing Blocklist)

If `payload.record.expression` is provided, it must not contain tokens associated with time-dependence, external I/O, or executable content. The driver rejects if any of the following substrings appear (case-insensitive):

```
time(, now(, clock, datetime, rand, random,
http, https, socket, file:, stdin, stdout,
import , __, exec, eval, open(, read(, write(
```

5 Refusal vs Rejection Semantics

5.1 Decision Types

The driver outputs one of:

- **ACCEPT**: Package is syntactically valid and meets frozen/grammar constraints.
- **REJECT**: Package violates objective rules (schema, budgets, frozen discipline, bounded grammar, hard domain invariants).
- **REFUSE**: Package is underdetermined or explicitly flagged as non-unique/ambiguous. This is *not* an internal error.

5.2 REFUSE Rules (Underdetermination Boundary)

Two refusal pathways exist:

1. Structural refusal (bounds missing):

If `payload.record.requires_bounds == true`, then `payload.record.bounds` must exist and contain min and max.

If missing or incomplete $\Rightarrow$ **REFUSE** with code `REFUSE_UNDERDETERMINED_MISSING_BOUNDS`.

2. Producer-declared refusal:

If `meta.flags.underdetermined == true` $\Rightarrow$ **REFUSE** with code `REFUSE_UNDERDETERMINED_OUTCO`

5.3 REJECT Rules (Objective Invalidity)

Hard rejection rules include:

- **Schema invalid** (missing required fields, invalid types, etc.) $\Rightarrow$ REJECT (or BAD_INPUT depending on invocation path).
- **Frozen blocks not frozen:** `protocol.frozen, comparator.frozen, bridge_rules.frozen` must be `true`.
- **Bounded grammar violations:** too many parameters, invalid parameter names, forbidden expression tokens, etc.
- **Domain invariant (minimal):** if `epsilon` (or `\u03b5`) is present in parameters, it must be numeric and within $[0, 1]$; otherwise REJECT with REJECT_EPSILON_OUT_OF_RANGE.

6 Canonicalization and Hashing

6.1 Canonical Record Scope

To ensure hash stability, the driver constructs a canonical view containing only stable fields:

```
canonical_view = {
  "message_type": ...,
  "package_id": ...,
  "producer": ...,
  "payload": ...,
  "provenance": ...,
  "protocol": ...,
  "comparator": ...,
  "bridge_rules": ...
}
```

Excluded by design:

- `created_at` (timestamps are excluded for stability)
- `integrity.*` (producer hashes are optional; avoid double-hashing)
- `meta` (debug/trace is not hash-relevant)

6.2 Quantization and Canonical Bytes

Canonicalization uses the project utilities:

- `canonical_record(..., q=DEFAULT_Q)` with `DEFAULT_Q = 1e-9`
- `canonical_bytes(..., q=DEFAULT_Q)`
- `sha256(canonical_bytes)`

7 Outputs

7.1 Decision Envelope (JSON)

The driver outputs a JSON decision envelope to **STDOUT**:

- `decision`: ACCEPT | REJECT | REFUSE
- `reasons`: list of reason objects (may be empty)
- `integrity`: hashes and versioning
- `references.audit.sha256`: hash over the canonical record
- `optional meta`: producer metadata passthrough
- `optional debug`: only when `meta.trace.debug == true`

7.2 Reason Object Format

Each reason entry is:

```
{
  "code": "<REASON_CODE>",
  "family": "<FAMILY>",
  "message": "<optional human string>",
  "path": "<optional JSON pointer>"
}
```

7.3 Exit Codes

The process exit code mirrors the decision:

Exit Code	Meaning
0	ACCEPT
2	REJECT
3	REFUSE
64	BAD_INPUT (empty input, invalid JSON, schema violation)
70	INTERNAL_ERROR (unexpected failure)

8 Reason Code Taxonomy (v1 Minimal Set)

8.1 REFUSE Families

Code	Trigger
REFUSE_UNDERDETERMINED_MISSING_BOUNDS	<code>requires_bounds=true</code> but bounds missing/incomplete
REFUSE_UNDERDETERMINED_OUTCOME	<code>meta.flags.underdetermined=true</code>

8.2 REJECT Families

Code	Trigger
REJECT_SCHEMA_MISSING_REQUIRED	Missing required field
REJECT_SCHEMA_TYPE_MISMATCH	Unsupported type/value in JSON structure
REJECT_CAPACITY_BUDGET_EXCEEDED	Any capacity budget exceeded
REJECT_NESTING_TOO_DEEP	Nesting depth > MAX_NESTING_DEPTH
REJECT_GRAMMAR_TOO_MANY_PARAMETERS	parameters count > MAX_PARAMETERS
REJECT_GRAMMAR_BAD_PARAM_NAME	parameter key violates regex
REJECT_GRAMMAR_EXPR_TOO_LONG	expression length > MAX_EXPRESSION_LEN
REJECT_GRAMMAR_PROGRAMMABLE_FORCING	forbidden token detected in expression
REJECT_PROTOCOL_INCOMPLETE	missing protocol required field
REJECT_PROJECTION_NOT_FROZEN	protocol.frozen != true
REJECT_COMPARATOR_NOT_FROZEN	comparator.frozen != true
REJECT_BRIDGE_RULE_CHANGED	bridge.rules.frozen != true
REJECT_AUTHORITY_FROZEN_MISSING_REVIEWER	frozen axiom but provenance.chain empty
REJECT_EPSILON_OUT_OF_RANGE	epsilon present but not in [0, 1]

9 Producer Guidance (XRAG Implementation Checklist)

To integrate with this driver, XRAG should:

1. Emit real freeze packages from its actual codepath (not hand-written examples).
2. Ensure protocol/comparator/bridge.rules are present and frozen=true.
3. Use candidate_axiom vs frozen_axiom to preserve authority separation.
4. When underdetermined, set either:
 - requires_bounds=true and provide bounds, or
 - meta.flags.underdetermined=true
5. Keep parameter naming and expression constraints within the bounded grammar.

10 Appendix: Minimal Example Output Envelopes

10.1 ACCEPT

```
{
  "decision": "ACCEPT",
  "reasons": [],
  "integrity": {
    "canonical_record_sha256": "<64 hex>",
    "driver_version": "xrag-driver/1.0.0",
    "contract_version": "freeze-ingest-contract/1.0.0",
    "context_hash": "<optional 64 hex>"
  },
  "references": { "audit": { "sha256": "<64 hex>" } }
}
```

10.2 REFUSE

```
{
  "decision": "REFUSE",
  "reasons": [{
    "code": "REFUSE_UNDERDETERMINED_MISSING_BOUNDS",
    "family": "REFUSE_UNDERDETERMINED",
    "message": "...",
    "path": "/payload/record/bounds"
  }],
  "integrity": { "...": "..." },
  "references": { "audit": { "sha256": "<64 hex>" } }
}
```

10.3 REJECT

```
{
  "decision": "REJECT",
  "reasons": [{
    "code": "REJECT_EPSILON_OUT_OF_RANGE",
    "family": "REJECT_DOMAIN",
    "message": "epsilon must be within [0,1]",
    "path": "/payload/record/parameters/epsilon"
  }],
  "integrity": { "...": "..." },
  "references": { "audit": { "sha256": "<64 hex>" } }
}
```